

main.rs

```
1 | mod config;
2 | mod error;
3 | mod file_scanner;
4 | mod pdf;
5 |
6 | #[cfg(feature = "syntax-highlighting")]
7 | mod highlighting;
8 |
9 | use clap::Parser;
10 | use std::path::PathBuf;
11 | use config::Config;
12 | use error::Result;
13 |
14 | /// Convert source code files to PDF with configurable headers and footers
15 | #[derive(Parser, Debug)]
16 | #[command(
17 |     name = "papercut",
18 |     version,
19 |     about = "Convert source code files to PDF with configurable headers and footers",
20 |     long_about = "Papercut is a CLI tool that converts source code files to PDFs based on a YAML
configuration file. \
21 |         It supports customizable headers, footers, syntax highlighting, and various page formatting
options."
22 | )]
23 | struct Args {
24 |     /// Path to the YAML configuration file
25 |     #[arg(short, long, value_name = "FILE")]
26 |     config: PathBuf,
27 |
28 |     /// Verbose output
29 |     #[arg(short, long)]
30 |     verbose: bool,
31 |
32 |     /// List available syntax highlighting themes (requires syntax-highlighting feature)
33 |     #[cfg(feature = "syntax-highlighting")]
34 |     #[arg(long)]
35 |     list_themes: bool,
36 | }
37 |
38 | fn main() -> Result<()> {
39 |     let args = Args::parse();
40 |
41 |     #[cfg(feature = "syntax-highlighting")]
42 |     if args.list_themes {
43 |         highlighting::list_themes();
44 |         return Ok(());
45 |     }
46 | }
```

```
47 | // Load configuration
48 | if args.verbose {
49 |     println!("Loading configuration from: {}", args.config.display());
50 | }
51 |
52 | let config = Config::from_file(&args.config)?;
53 |
54 | if args.verbose {
55 |     println!("Configuration loaded successfully");
56 |     println!("Output mode: {:?}", config.output.mode);
57 |     println!("File patterns specified: {}", config.files.len());
58 |     println!("Files matched: {}", config.expanded_files.len());
59 | }
60 |
61 | // Create output directory if it doesn't exist
62 | std::fs::create_dir_all(&config.output.directory)?;
63 |
64 | if args.verbose {
65 |     println!("Output directory: {}", config.output.directory.display());
66 | }
67 |
68 | // Generate PDF(s)
69 | pdf::generate(config, args.verbose)?;
70 |
71 | if args.verbose {
72 |     println!("PDF generation completed successfully!");
73 | }
74 |
75 | Ok(())
76 | }
```

```
1 | use serde::{Deserialize, Serialize};
2 | use std::path::PathBuf;
3 | use crate::error::{PapercutError, Result};
4 |
5 | #[derive(Debug, Deserialize, Serialize, Clone)]
6 | pub struct Config {
7 |     pub output: OutputConfig,
8 |     pub files: Vec<FileEntry>,
9 |     /// Expanded file list (populated after pattern expansion)
10 |    #[serde(skip)]
11 |    pub expanded_files: Vec<ExpandedFileEntry>,
12 |    #[serde(default)]
13 |    pub syntax_highlighting: SyntaxHighlightingConfig,
14 |    #[serde(default)]
15 |    pub page: PageConfig,
16 |    #[serde(default)]
17 |    pub header: HeaderFooterConfig,
18 |    #[serde(default)]
19 |    pub footer: HeaderFooterConfig,
20 |    #[serde(default)]
21 |    pub styling: StylingConfig,
22 |    #[serde(default)]
23 |    pub metadata: MetadataConfig,
24 | }
25 |
26 | /// Expanded file entry after pattern matching
27 | #[derive(Debug, Clone)]
28 | pub struct ExpandedFileEntry {
29 |     pub path: PathBuf,
30 |     pub title: Option<String>,
31 | }
32 |
33 | #[derive(Debug, Deserialize, Serialize, Clone)]
34 | pub struct OutputConfig {
35 |     /// "single" for one combined PDF, "multiple" for one PDF per file
36 |     pub mode: OutputMode,
37 |     /// Output directory path
38 |     #[serde(default = "default_output_dir")]
39 |     pub directory: PathBuf,
40 |     /// Filename for single mode (optional, defaults to "output.pdf")
41 |     #[serde(default = "default_output_filename")]
42 |     pub filename: String,
43 | }
44 |
45 | #[derive(Debug, Deserialize, Serialize, Clone, PartialEq)]
46 | #[serde(rename_all = "lowercase")]
47 | pub enum OutputMode {
48 |     Single,
```

```
49 |     Multiple,
50 | }
51 |
52 | #[derive(Debug, Deserialize, Serialize, Clone)]
53 | pub struct FileEntry {
54 |     pub path: PathBuf,
55 |     /// Optional custom title for this file in the PDF
56 |     #[serde(default)]
57 |     pub title: Option<String>,
58 |     /// File types to include (by extension, e.g., ["rs", "py", "js"])
59 |     /// If empty, all file types are included
60 |     #[serde(default)]
61 |     pub include_types: Vec<String>,
62 |     /// Patterns to exclude (e.g., ["*.test.rs", "target/**"])
63 |     /// Uses glob pattern syntax
64 |     #[serde(default)]
65 |     pub exclude: Vec<String>,
66 | }
67 |
68 | #[derive(Debug, Deserialize, Serialize, Clone)]
69 | pub struct SyntaxHighlightingConfig {
70 |     #[serde(default = "default_false")]
71 |     pub enabled: bool,
72 |     /// Theme name (e.g., "base16-ocean.dark", "InspiredGitHub")
73 |     #[serde(default = "default_theme")]
74 |     pub theme: String,
75 | }
76 |
77 | impl Default for SyntaxHighlightingConfig {
78 |     fn default() -> Self {
79 |         Self {
80 |             enabled: false,
81 |             theme: default_theme(),
82 |         }
83 |     }
84 | }
85 |
86 | #[derive(Debug, Deserialize, Serialize, Clone)]
87 | pub struct PageConfig {
88 |     /// Page size: "A4", "Letter", "Legal"
89 |     #[serde(default = "default_page_size")]
90 |     pub size: PageSize,
91 |     /// Margins in centimeters
92 |     #[serde(default)]
93 |     pub margins: MarginsConfig,
94 |     /// Font size for code content
95 |     #[serde(default = "default_font_size")]
96 |     pub font_size: u8,
97 |     /// Show line numbers
98 |     #[serde(default = "default_true")]
99 |     pub line_numbers: bool,
```

```

100 |     /// Line spacing multiplier
101 |     #[serde(default = "default_line_spacing")]
102 |     pub line_spacing: f32,
103 | }
104 |
105 | impl Default for PageConfig {
106 |     fn default() -> Self {
107 |         Self {
108 |             size: default_page_size(),
109 |             margins: MarginsConfig::default(),
110 |             font_size: default_font_size(),
111 |             line_numbers: true,
112 |             line_spacing: default_line_spacing(),
113 |         }
114 |     }
115 | }
116 |
117 | #[derive(Debug, Deserialize, Serialize, Clone, PartialEq)]
118 | pub enum PageSize {
119 |     A4,
120 |     Letter,
121 |     Legal,
122 | }
123 |
124 | #[derive(Debug, Deserialize, Serialize, Clone)]
125 | pub struct MarginsConfig {
126 |     #[serde(default = "default_margin_top")]
127 |     pub top: f32,
128 |     #[serde(default = "default_margin_bottom")]
129 |     pub bottom: f32,
130 |     #[serde(default = "default_margin_left")]
131 |     pub left: f32,
132 |     #[serde(default = "default_margin_right")]
133 |     pub right: f32,
134 | }
135 |
136 | impl Default for MarginsConfig {
137 |     fn default() -> Self {
138 |         Self {
139 |             top: default_margin_top(),
140 |             bottom: default_margin_bottom(),
141 |             left: default_margin_left(),
142 |             right: default_margin_right(),
143 |         }
144 |     }
145 | }
146 |
147 | #[derive(Debug, Deserialize, Serialize, Clone)]
148 | pub struct HeaderFooterConfig {
149 |     #[serde(default = "default_false")]
150 |     pub enabled: bool,

```

```

151 |     /// Text to display on the left (supports variables: {page}, {total}, {filename}, {date})
152 |     #[serde(default)]
153 |     pub left: String,
154 |     /// Text to display in the center
155 |     #[serde(default)]
156 |     pub center: String,
157 |     /// Text to display on the right
158 |     #[serde(default)]
159 |     pub right: String,
160 |     /// Font size for header/footer
161 |     #[serde(default = "default_header_footer_font_size")]
162 |     pub font_size: u8,
163 | }
164 |
165 | impl Default for HeaderFooterConfig {
166 |     fn default() -> Self {
167 |         Self {
168 |             enabled: false,
169 |             left: String::new(),
170 |             center: String::new(),
171 |             right: String::new(),
172 |             font_size: default_header_footer_font_size(),
173 |         }
174 |     }
175 | }
176 |
177 | #[derive(Debug, Deserialize, Serialize, Clone)]
178 | pub struct StylingConfig {
179 |     /// Font family for code: "monospace", "courier", "dejavu"
180 |     #[serde(default = "default_font_family")]
181 |     pub font_family: FontFamily,
182 |     /// Background color for code blocks (hex format, e.g., "#ffffff")
183 |     #[serde(default = "default_background_color")]
184 |     pub background_color: String,
185 |     /// Text color (hex format, e.g., "#000000")
186 |     #[serde(default = "default_text_color")]
187 |     pub text_color: String,
188 |     /// Line number color (hex format)
189 |     #[serde(default = "default_line_number_color")]
190 |     pub line_number_color: String,
191 | }
192 |
193 | impl Default for StylingConfig {
194 |     fn default() -> Self {
195 |         Self {
196 |             font_family: default_font_family(),
197 |             background_color: default_background_color(),
198 |             text_color: default_text_color(),
199 |             line_number_color: default_line_number_color(),
200 |         }
201 |     }

```

```

202 | }
203 |
204 | #[derive(Debug, Deserialize, Serialize, Clone, PartialEq)]
205 | pub enum FontFamily {
206 |     #[serde(rename = "monospace")]
207 |     Monospace,
208 |     #[serde(rename = "courier")]
209 |     Courier,
210 |     #[serde(rename = "dejavu")]
211 |     DejaVu,
212 | }
213 |
214 | #[derive(Debug, Deserialize, Serialize, Clone)]
215 | pub struct MetadataConfig {
216 |     #[serde(default)]
217 |     pub title: String,
218 |     #[serde(default)]
219 |     pub author: String,
220 |     #[serde(default)]
221 |     pub subject: String,
222 |     #[serde(default)]
223 |     pub keywords: Vec<String>,
224 | }
225 |
226 | impl Default for MetadataConfig {
227 |     fn default() -> Self {
228 |         Self {
229 |             title: String::new(),
230 |             author: String::new(),
231 |             subject: String::new(),
232 |             keywords: Vec::new(),
233 |         }
234 |     }
235 | }
236 |
237 | // Default value functions
238 | fn default_output_dir() -> PathBuf {
239 |     PathBuf::from("./output")
240 | }
241 |
242 | fn default_output_filename() -> String {
243 |     "output.pdf".to_string()
244 | }
245 |
246 | fn default_false() -> bool {
247 |     false
248 | }
249 |
250 | fn default_true() -> bool {
251 |     true
252 | }

```

```
253 |  
254 | fn default_theme() -> String {  
255 |     "base16-ocean.dark".to_string()  
256 | }  
257 |  
258 | fn default_page_size() -> PageSize {  
259 |     PageSize::A4  
260 | }  
261 |  
262 | fn default_font_size() -> u8 {  
263 |     10  
264 | }  
265 |  
266 | fn default_line_spacing() -> f32 {  
267 |     1.2  
268 | }  
269 |  
270 | fn default_margin_top() -> f32 {  
271 |     2.5  
272 | }  
273 |  
274 | fn default_margin_bottom() -> f32 {  
275 |     2.5  
276 | }  
277 |  
278 | fn default_margin_left() -> f32 {  
279 |     2.0  
280 | }  
281 |  
282 | fn default_margin_right() -> f32 {  
283 |     2.0  
284 | }  
285 |  
286 | fn default_header_footer_font_size() -> u8 {  
287 |     8  
288 | }  
289 |  
290 | fn default_font_family() -> FontFamily {  
291 |     FontFamily::Monospace  
292 | }  
293 |  
294 | fn default_background_color() -> String {  
295 |     "#ffffff".to_string()  
296 | }  
297 |  
298 | fn default_text_color() -> String {  
299 |     "#000000".to_string()  
300 | }  
301 |  
302 | fn default_line_number_color() -> String {  
303 |     "#888888".to_string()
```

```

304 | }
305 |
306 | impl Config {
307 |     /// Load configuration from a YAML file
308 |     pub fn from_file(path: &PathBuf) -> Result<Self> {
309 |         let contents = std::fs::read_to_string(path)
310 |             .map_err(|e| PapercutError::Config(format!("Failed to read config file: {}", e)))?;
311 |
312 |         let mut config: Config = serde_yaml::from_str(&contents)?;
313 |
314 |         // Expand file patterns before validation
315 |         config.expand_file_patterns()?;
316 |         config.validate()?;
317 |
318 |         Ok(config)
319 |     }
320 |
321 |     /// Expand file patterns and populate expanded_files
322 |     fn expand_file_patterns(&mut self) -> Result<()> {
323 |         use crate::file_scanner;
324 |
325 |         let mut expanded_files = Vec::new();
326 |
327 |         for file_entry in &self.files {
328 |             let files = file_scanner::expand_file_patterns(
329 |                 &file_entry.path,
330 |                 &file_entry.include_types,
331 |                 &file_entry.exclude,
332 |             )?;
333 |
334 |             // Add each expanded file with the original title (if provided)
335 |             for file_path in files {
336 |                 expanded_files.push(ExpandedFileEntry {
337 |                     path: file_path,
338 |                     title: file_entry.title.clone(),
339 |                 });
340 |             }
341 |         }
342 |
343 |         self.expanded_files = expanded_files;
344 |         Ok(())
345 |     }
346 |
347 |     /// Validate the configuration
348 |     fn validate(&self) -> Result<()> {
349 |         // Validate that we have at least one file specification
350 |         if self.files.is_empty() {
351 |             return Err(PapercutError::InvalidConfig(
352 |                 "No files specified in configuration".to_string()
353 |             ));
354 |         }

```

```
355 |  
356 | // Validate that pattern expansion resulted in at least one file  
357 | if self.expanded_files.is_empty() {  
358 |     return Err(PapercutError::InvalidConfig(  
359 |         "No files matched the specified patterns".to_string()  
360 |     ));  
361 | }  
362 |  
363 | // Validate that all expanded files exist and are readable  
364 | for file_entry in &self.expanded_files {  
365 |     if !file_entry.path.exists() {  
366 |         return Err(PapercutError::FileNotFound(  
367 |             file_entry.path.display().to_string()  
368 |         ));  
369 |     }  
370 |     if !file_entry.path.is_file() {  
371 |         return Err(PapercutError::InvalidConfig(  
372 |             format!("{}", file_entry.path.display())  
373 |         ));  
374 |     }  
375 | }  
376 |  
377 | // Validate margin values  
378 | if self.page.margins.top < 0.0 || self.page.margins.bottom < 0.0 ||  
379 |     self.page.margins.left < 0.0 || self.page.margins.right < 0.0 {  
380 |     return Err(PapercutError::InvalidConfig(  
381 |         "Margins must be non-negative".to_string()  
382 |     ));  
383 | }  
384 |  
385 | // Validate font sizes  
386 | if self.page.font_size == 0 {  
387 |     return Err(PapercutError::InvalidConfig(  
388 |         "Font size must be greater than 0".to_string()  
389 |     ));  
390 | }  
391 |  
392 | Ok(())  
393 | }  
394 | }
```

```
1 | # Papercut
2 |
3 | Convert source code files to PDF with configurable headers, footers, and formatting.
4 |
5 | ## Quick Start
6 |
7 | ### Installation
8 |
9 | ```bash
10 | git clone https://github.com/yourusername/papercut.git
11 | cd papercut
12 | cargo install --path .
13 | ```
14 |
15 | ### Basic Usage
16 |
17 | 1. Create a configuration file `config.yaml`:
18 |
19 | ```yaml
20 | output:
21 |   mode: single
22 |   directory: ./output
23 |   filename: code.pdf
24 |
25 | files:
26 |   - path: src/main.rs
27 |   - path: src/lib.rs
28 | ```
29 |
30 | 2. Generate PDF:
31 |
32 | ```bash
33 | papercut --config config.yaml
34 | ```
35 |
36 | ### Features
37 |
38 | - Directory Scanning & Glob Patterns: Use wildcards (`*.rs`, `src/**/*.py`) and scan entire directories
39 | - File Filtering: Include/exclude specific file types and patterns
40 | - Single or Multiple PDFs: Combine all files or create separate PDFs
41 | - Syntax Highlighting: Optional syntax highlighting with multiple themes
42 | - Customizable Headers/Footers: Add page numbers, dates, filenames, and custom text
43 | - Flexible Formatting: Configure page size, margins, fonts, and colors
44 | - PDF Metadata: Set title, author, subject, and keywords
45 |
46 | ### Example Configurations
47 |
48 | See the `examples/` directory:
```

```
49 |
50 | - `minimal_config.yaml` - Bare minimum configuration
51 | - `full_config.yaml` - All available options with comments
52 | - `release_config.yaml` - Public release template
53 | - `patterns_config.yaml` - Directory scanning and pattern matching examples
54 |
55 | ### Documentation
56 |
57 | Complete documentation is available in the `docs/` directory:
58 |
59 | - [Configuration Reference](docs/configuration.md) - All configuration options
60 | - [Usage Guide](docs/usage.md) - CLI usage and workflows
61 | - [Styling Guide](docs/styling.md) - Advanced styling options
62 | - [Examples](docs/examples.md) - Real-world use cases
63 |
64 | ### List Available Themes
65 |
66 | ```bash
67 | papercut --list-themes
68 | ```
69 |
70 | ### Help
71 |
72 | ```bash
73 | papercut --help
74 | ```
75 |
76 | ## License
77 |
78 | This project is licensed under the GNU General Public License v3.0 - see the [LICENSE](LICENSE) file
for details.
```